

CREATING AI-AGENTS WITH TRANSFORMER MODEL AND MCP TO PROVIDE TREATMENT/DIAGNOSIS OF COMMON ILLNESSES

Dennis Tislin

5/20/2026

**Computer Systems Lab, Dr. Yilmaz, Period 7
TJ Star**

PROBLEM



- Healthcare premiums and costs have become exceedingly high
 - Continues to increase
 - Medicare spending grew 8.1% to \$1,029.8 billion in 2023 [8]
 - Medicaid spending grew 7.9% to \$871.7 billion in 2023 [8]
- Availability of healthcare has decreased, and many struggle to find optimal treatment
- Treatment has also become more expensive over time
- Overall inefficiency of healthcare in the U.S. has greatly increased hospital spending

BACKGROUND

- To simplify healthcare options for illnesses, an LLM can be created or tuned to provide accessible diagnosis/treatment
- By using Transformer Model Neural Networks to train a new (or previously trained) LLM on the symptoms and treatment of a range of illnesses, a more accessible healthcare option could be created
- By potentially utilizing MCP servers, an AI-agent could execute actions/methods of treatment based on LLM intuition

BACKGROUND

- Large Language Model (LLM)
 - Having neural networks trained on datasets, LLMs have the ability to understand/interact with human language
 - Different types of models excel at different tasks: a Transformer Model excels at natural language processing (NLP) and sequential data (data with points dependent on one another)
- AI Agent
 - An AI Agent is the combination of an LLM, MCP Client, and MCP Server
 - Agents are able to conduct actions to pursue a goal
- MCP Client
 - An MCP Client is a software component connecting an LLM to an MCP Server, truly creating an AI Agent
- Model Context Protocol (MCP)
 - MCP is an extension of an AI-Agent (a program that uses an LLM to autonomously learn and make decisions)
 - It allows the linkage between an MCP Client and an MCP server granting executable actions on command

BACKGROUND (DELETE SLIDE)

- Current AI Models struggle to accurately diagnose illnesses
- Models may only provide suggestions of actions, rather than attempt to perform any acts of aid

OTHER SOLUTIONS (DELETE SLIDE)

- Predictive Modelling has been utilized to calculate the risk of patients' development of disease, enabling more effective treatment plans
- Medical Image Analysis has been utilized to increase the speed/accuracy of disease diagnosis
- AI devices have been created to monitor patient data outside of institutions to periodically track their health

OTHER SOLUTIONS

➤ **Buoy Health**

- AI symptom checker that uses a set of questions to determine causes of symptoms, as well as possible care options [15]
- Possesses no ability to provide direct diagnosis

➤ **DxGPT**

- Unable to communicate with user, input consists only of variables: age, gender, symptoms, and when the symptoms began
- In a study to evaluate the utility of DxGPT as a support tool for diagnosis of both common and rare diseases, the AI struggled to match human accuracy [9]
 - Accuracies for common diseases were higher for human clinicians (Clinicians: 79%, DxGPT: 71%)

OTHER SOLUTIONS

➤ ChatGPT

- Able to provide causes behind symptoms, as well as indicators, but struggles to provide direct diagnosis
 - In a study testing ChatGPT 4.0's accuracy on the USMLE STEP 2 CK test (focusing on diagnosis and patient management in internal medicine, pediatrics, and surgery) ChatGPT achieved an accuracy 87.2% [14]
- Trained on data from textbooks, research papers, health websites (like NIH, CDC, WHO) and medical encyclopedias [14]
 - Lacks training on datasets that provide discrete symptoms

NOVELTY

- Previous applications contribute to a smaller population
 - Predictive Modelling has been used for more rare diseases/illnesses [15]
 - Cancer, neurological diseases, and cardiovascular diseases
- Past innovation has been narrowly focused on a specific issue/rare diseases mentioned above
- Due to narrow-minded focus on specific issues, past innovation struggles to improve healthcare affordability and availability

NOVELTY

- My innovation would provide a direct diagnosis and perform helpful actions
 - Buoy Health lacks such advancement
- Application of AI-Agent in healthcare
 - Usage of MCP to conduct a direct act of aide
 - Contacting pharmacies
 - Accessing specific information through the web
- Uncommon focus on general illnesses: accounting for a large population

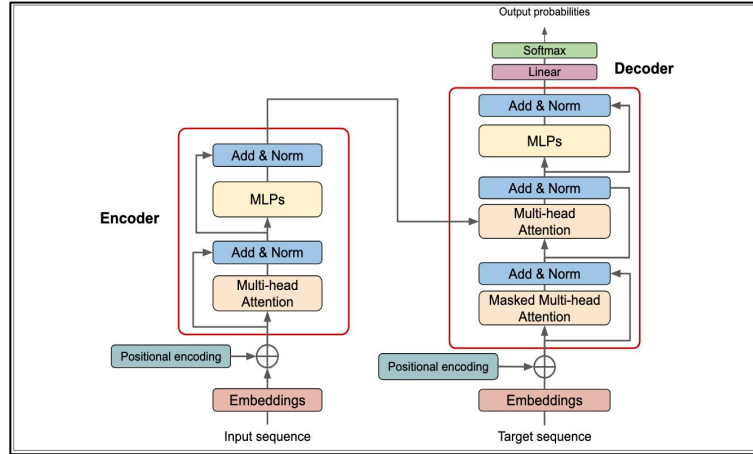
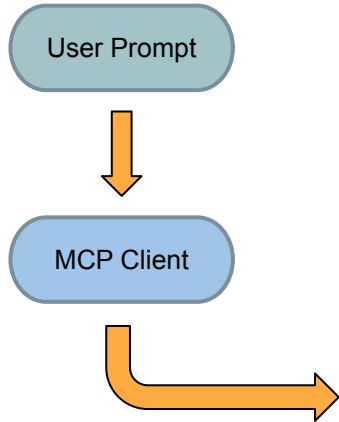
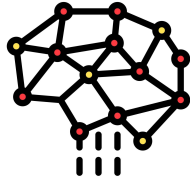
IMPACT

- Citizens with few healthcare resources will gain an easily accessible possibility that provides accurate diagnosis and treatment plans
- A large population will be positively impacted, since a wide range of diseases/illnesses will become diagnosable and treatable with minimal effort
- Such a resource would not include user costs, allowing lower-income populations a free resource

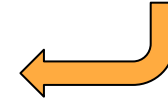
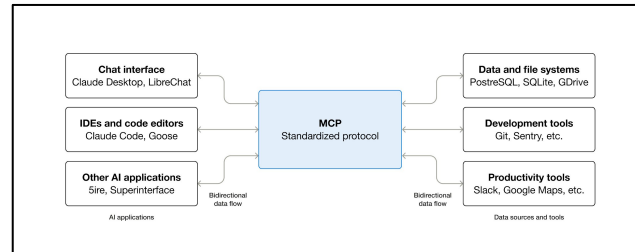
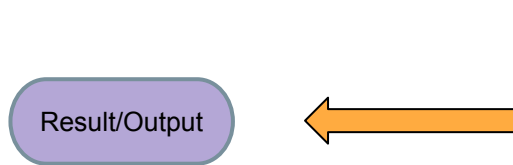
WHY IS THIS BETTER?

- Advancements in such a field have undermined the importance of popular illnesses/diseases, and availability of treatment, which remains as an ongoing struggle
- The extension of possible treatments through MCP could execute meaningful actions for the user: a very limited advancement seen in modern LLMs
- A direct diagnosis for symptoms, rather than explaining their causes, would be provided
- Transforms a complex, previously created LLM through fine-tuning on datasets to a refined version possessing greater intelligence

AI-AGENT/SYSTEM ARCHITECTURE



```
{
  "mcpServers": {
    "memory": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"]
    }
  }
}
```



METHOD

- By using datasets from Kaggle, an LLM (previously trained) would be trained an appropriate number of epochs
- Tensorflow and Pytorch frameworks allow for such manipulations by utilizing APIs to program training loops and Neural Networks–Transformer Models in my instance, since sequential data is involved [2]
- MCP servers could connect LLM applications to provide an array of executable actions [7]

SELECTED LARGE LANGUAGE MODEL (LLM)

- While GPT-4 was the intended fine-tuning Transformer Model, it was not available
 - As a Result, GPT-2 was chosen and Llama3 OpenBioLLM
- GPT-2 and Llama3 have different sizes (parameter amounts)
 - Parameters are internal variables within a neural network that a model adjusts during training/optimization
 - Due to limited access, a small size of GPT-2 and Llama3 were chosen
 - Difficult to acquire fine-tuning resources (many GPUs + lots of RAM/memory)
 - The GPT-2 model has only 124m parameters
 - The Llama3 model has ~8b parameters
 - Already fine tuned to a domain of interest: healthcare

DATASET CONVERSION

- Kaggle dataset [5] → Hugging Face dataset
 - Hugging Face models, such as the one I am using, require datasets on the HF Hub (open source resources) in order to be fine tuned
 - From CSV (comma separated values) file to a parquet (HF storage format)
 - reduced memory requirement, fast data retrieval and filtering, and efficient storage
- Converted via adjusted python script from GitHub [17]
 - nateraw/huggingface-datasets-converter [17]
 - Outdated API function caused long term problems: *'KaggleApi' object has no attribute 'metadata_get'*
 - Modified code to work around this issue
 - Included 80%-20% data split
 - Train: 3936 rows/dictionaries
 - Test: 984 rows/dictionaries
- Script uploads converted dataset to HF Hub

INITIAL KAGGLE DATASET

Kaggle (Disease and Symptoms)

△ Disease	△ Symptom_1	△ Symptom_2	△ Symptom_3
Disease name	Symptom of the disease	Symptom of the disease	Symptom of the disease
41 unique values	vomiting 17% fatigue 14% Other (3408) 69%	vomiting 18% fatigue 8% Other (3648) 74%	fatigue 15% high_fever 7% Other (3870) 79%
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	skin_rash	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	skin_rash	dischromic _patches
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	skin_rash	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	skin_rash	dischromic _patches
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	itching	skin_rash	nodal_skin_eruptions

CONVERTED KAGGLE DATASETS

Disease and Symptoms on Kaggle

△ Disease	△ Symptom_1		△ Symptom_2		△ Symptom_3	
Disease name	Symptom of the disease		Symptom of the disease		Symptom of the disease	
41 unique values	vomiting	17%	vomiting	18%	fatigue	15%
	fatigue	14%	fatigue	8%	high_fever	7%
	Other (3408)	69%	Other (3648)	74%	Other (3870)	79%
Fungal infection	itching		skin_rash		nodal_skin_eruptions	
Fungal infection	skin_rash		nodal_skin_eruptions		dischromic _patches	
Fungal infection	itching		nodal_skin_eruptions		dischromic _patches	
Fungal infection	itching		skin_rash		dischromic _patches	
Fungal infection	itching		skin_rash		nodal_skin_eruptions	
Fungal infection	skin_rash		nodal_skin_eruptions		dischromic _patches	
Fungal infection	itching		nodal_skin_eruptions		dischromic _patches	
Fungal infection	itching		skin_rash		dischromic _patches	
Fungal infection	itching		skin_rash		nodal_skin_eruptions	
Fungal infection	itching		skin_rash		nodal_skin_eruptions	



Example from Train split

disease	symptoms
string · classes	list · lengths
	
Fungal infection	["itching", "skin_rash", "nodal_skin_eruptions", "dischromic _patches"]
Psoriasis	["skin_rash", "joint_pain", "skin_peeling", "silver_like_dusting", "small_dents_in_nails", ...]
Alcoholic hepatitis	["vomiting", "yellowish_skin", "abdominal_pain", "swelling_of_stomach", "history_of_alcohol_consumption", ...]
Hypoglycemia	["vomiting", "fatigue", "anxiety", "sweating", "headache", "nausea", "blurred_and_distorted_vision", ...]
Pneumonia	["chills", "fatigue", "cough", "high_fever", "breathlessness", "sweating", "malaise", "phlegm", ...]
Typhoid	["chills", "vomiting", "fatigue", "high_fever", "headache", "nausea", "constipation", "abdominal_pain", ...]
Fungal infection	["itching", "skin_rash", "nodal_skin_eruptions", "dischromic _patches"]
Bronchial Asthma	["fatigue", "cough", "high_fever", "breathlessness", "family_history", "mucoid_sputum"]
Osteoarthritis	["joint_pain", "neck_pain", "knee_pain", "hip_joint_pain", "swelling_joints", "painful_walking"]
(vertigo) Paroymsal Positional Vertigo	["headache", "nausea", "spinning_movements", "loss_of_balance", "unsteadiness"]

DATASET CONVERSION PROCESS

```
1 transformers
2 datasets
3 torch
4 torchaudio
5 torchvision
6 accelerate>=0.26.0
7 huggingface-hub
```

requirements.txt

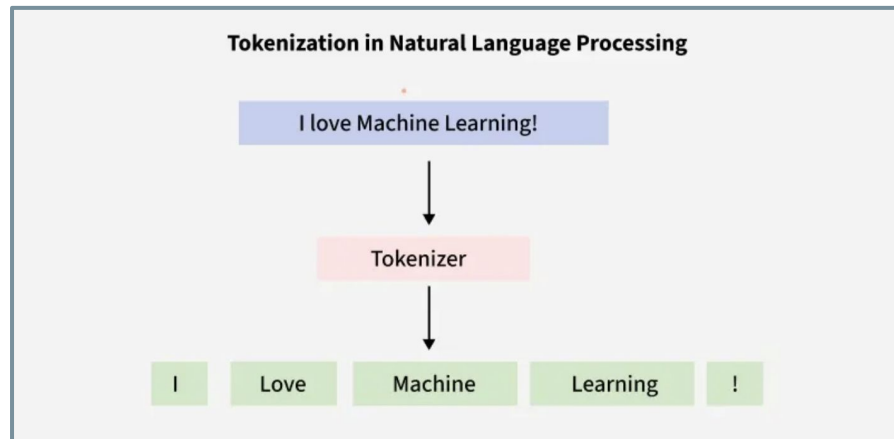
```
> hf auth login
```

Name	Value	Last Refreshed Date	Last Used Date	Permissions
supersecrettoken	hf_...xkQO	Oct 29	3 days ago	WRITE

MagazinePuma/Disease_and_symptoms

PREPROCESSING DATA

- Utilizing Disease and Symptoms Kaggle dataset
- Splitting Kaggle datasets into training/testing arguments
- NLP → Tokenization
 - Hugging Face is an open source AI community containing many models and datasets
 - Hugging Face uses AutoTokenizer (Pytorch) or Keras Tokenizer (Tensorflow) API
 - Splitting sentences to words (tokens) and assigning numerical values and word embedding vectors to help the model identify patterns (Word Tokenization)
 - No need for initial normalization



`ex["text"][0]`

I have itching, skin rash, nodal skin eruptions, dischromic patches. I might be experiencing Fungal infection

PREPROCESSING DATA CODE (GPT-2)

```
def initialize(ex):
    disease = ex["disease"]
    symptoms = ex["symptoms"]
    eng_symptoms = []

    for symptom in symptoms:

        if "_" in symptom:
            symptom = symptom.replace("_", " ")

        eng_symptoms.append(symptom)

    csv_symptoms = ", ".join(eng_symptoms)
    ex["text"] = f"I have {csv_symptoms}. I might be experiencing {disease}"
    # ex["text"] = f"The symptoms of {disease} are {csv_symptoms}"
    # ex["text"] = f"Disease: {disease}\nSymptoms: {csv_symptoms}\n"
    return ex

dataset = dataset.map(initialize)
```

```
def tokenize(batch):
    return tokenizer(batch["text"], padding="max_length", truncation=True)

tokenizer.pad_token = tokenizer.eos_token
tokenized = dataset.map(tokenize, batched=True)
```

```
ex["text"][0]
```

[40, 423, 48140, 11, 4168, 28509, 11, 18666, 282, 4168, 13724, 8544, 11, 595, 28663, 291, 220, 16082, 13, 314, 1244, 307, 13456, 376, 2150, 282, 10280, 50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256, 5]

PREPROCESSING DATA CODE (LLAMA3 OPENBIOLLM)

```
def initialize(ex):
    disease = ex["disease"]
    symptoms = ex["symptoms"]
    eng_symptoms = []

    for symptom in symptoms:

        if "_" in symptom:
            symptom = symptom.replace("_", " ")

        eng_symptoms.append(symptom)

    csv_symptoms = ", ".join(eng_symptoms)
    ex["text"] = f"<|user|>\nI have {csv_symptoms}. What disease might I have?\n<|assistant|>\nYou might have {disease}"
    return ex

dataset = dataset.map(initialize)
```



```
def tokenize(batch):
    ret = tokenizer(batch["text"], padding="max_length", truncation=True, max_length=256)
    ret["labels"] = ret["input_ids"].copy()

    return ret
```

TRAINING ON THE NEURAL NETWORK

- After tokenization, a Transformer Model Neural Network must be made/used
 - Dealing with Sequential Data
 - Excels at language translation/generation in NLP and weighs importance of tokens (input)
 - Attention Mechanism → uses mathematical techniques to detect relationships/dependencies and importance of tokens
- Positional Encoders tag inputs/outputs with Attention units to create an algebraic map signifying relationships
- Parallel processing of elements allows for the recognition of patterns, and optimizes speed

TRAINING (ATTENTION)

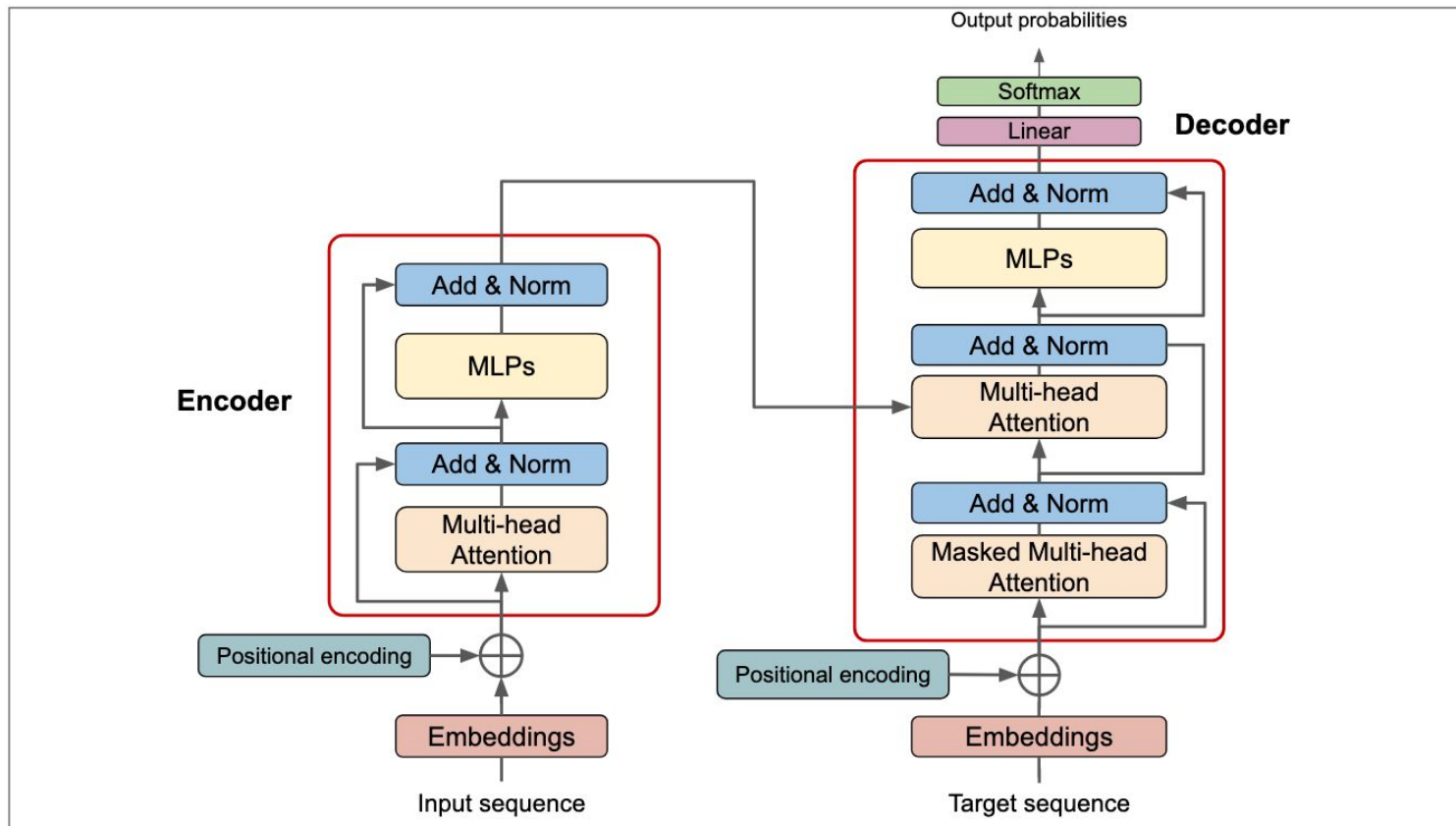
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{K^T Q}{\sqrt{d_k}}\right) V$$

- The Attention mechanism allows the Transformer to understand English and relationships between words
 - Ex: “I have itching, skin rash, etc.... I might be experiencing Fungal infection”
- Single head attention
 - One word embedding vector of a sequence undergoes this mechanism
 - Word embeddings **uniquely** define a word
 - These vectors function in an extremely large 3d space
 - Positional encoding tags inform the Transformer on a token's position
 - These word embeddings have a **“Q” (query), “K” (Key), and “V” (value) vector** [6]
 - “Q” vector provides info on what the word embedding vector is looking for
 - “K” vector provides info on the word embedding vector
 - “V” vector holds relevant info for other tokens, relative to the word embedding vector
 - Q, K, and V are updated as Transformer looks at each token
 - Every word embedding vector of a sequence has a unique Q, K, V value in a global matrix that signifies relationships between tokens

TRAINING (ATTENTION)

- Models look at multiple sequences in parallel, called Multi-Head Attention
 - Single Head Attention of multiple sequences
- Commonly, models split sequences into sub-sequences to better understand relationships between words
 - Ex: “I have itching, skin rash...”
 - Ex: “I have itching”
 - Ex: “I have”
 - In order to prevent the Transformer from already knowing what a sub-sequence looks like based on a longer version of that sub-sequence, the Transformer **masks** tokens in longer sequences that appear further in the original sequence
 - This process is called **Masked Multi-Head Attention**
- Multi-Layer Perceptron (MLP)
 - Same as Forward Feed Neural Network
 - Word embedding vectors are updated + normalized
 - Q, K, and V matrix is also updated

TRANSFORMER MODEL ARCHITECTURE (*ATTENTION IS ALL YOU NEED*)



TUNING CODE (GPT2)

```
training_args = TrainingArguments(  
    output_dir="./fine_tuned_gpt2",  
    overwrite_output_dir=True,  
    num_train_epochs=3,  
    per_device_train_batch_size=4,  
    save_steps=500,  
    save_total_limit=2,  
    logging_steps=100  
)
```

Arguments



```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized["train"],  
    eval_dataset=tokenized["test"],  
    data_collator=data_collator,  
    processing_class=tokenizer  
)
```

Hyperparameters



```
trainer.train()
```

TUNING CODE (LLAMA3 OPENBIOLLM)

```
training_args = TrainingArguments(  
    output_dir="./fine_tuned_llama8B",  
    num_train_epochs=3,  
    per_device_train_batch_size=1,  
    learning_rate=2e-5,  
    save_steps=500,  
    save_total_limit=2,  
    logging_steps=100,  
    eval_strategy="epoch"  
)
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized["train"],  
    eval_dataset=tokenized["test"],  
    processing_class=tokenizer  
)  
  
trainer.train()
```

LLAMA3 OPENBIOLLM TUNING OPTIMIZATIONS

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

bnb = BitsAndBytesConfig(load_in_4bit=True)

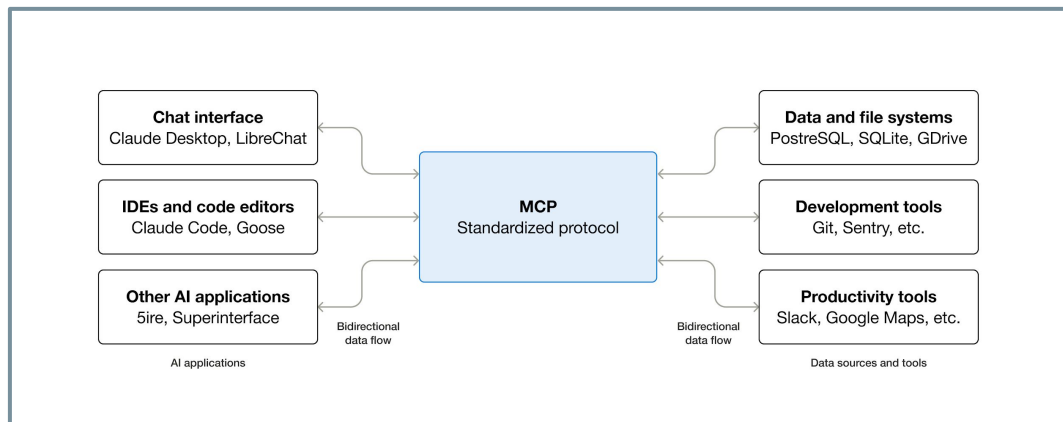
model = AutoModelForCausalLM.from_pretrained(
    "aaditya/Llama3-OpenBioLLM-8B",
    quantization_config=bnb,
    device_map="auto"
)

model = get_peft_model(model, lora_config)
```

- Because the model is large, fine tuning the model without optimizations would require more hardware capabilities than provided
- Weights are normally represented in 32 bit floats; QLoRA allows us to load the weights in 4 bit, which saves lots of memory during tuning
- “r” is the size of the smaller matrices adjusted during training, instead of the weights

MODEL CONTEXT PROTOCOL (MCP)

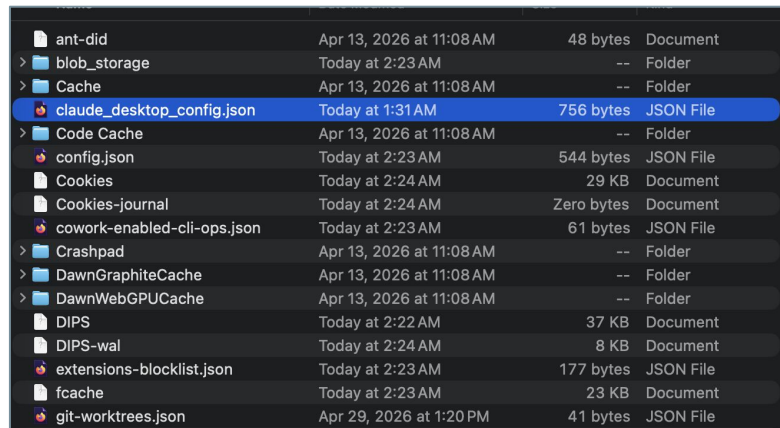
- MCP is an open source standard for connecting AI applications to external systems
- After successfully training/testing the model, MCP can be used to give the model executable actions
- MCP clients connect LLMs to MCP servers that grant tools or data sources
 - By requesting actions to the server, the client allows the model to conduct actions through tools from the server



MODEL CONTEXT PROTOCOL (MCP)

- MCP servers transform models into AI agents with the addition of executable actions on request
- An MCP Client, such as Claude Desktop, are compatible with MCP and can connect to servers
- Servers can be found on open source websites, such as GitHub
- Json files allow for the configuration of MCP servers
 - Simple Claude AI configuration to attain Git server:

```
{  
  "mcpServers": {  
    "disease-agent": {  
      "command": "/opt/homebrew/Caskroom/miniconda/base/envs/new_env/bin/python3",  
      "args": [  
        "/Users/dennis/Desktop/Research/Tensorflow/Mcp_server.py"  
      ]  
    }  
  }  
},
```



ant-did	Apr 13, 2026 at 11:08 AM	48 bytes	Document
> blob_storage	Today at 2:23 AM	--	Folder
> Cache	Apr 13, 2026 at 11:08 AM	--	Folder
claude_desktop_config.json	Today at 1:31 AM	756 bytes	JSON File
> Code Cache	Apr 13, 2026 at 11:08 AM	--	Folder
config.json	Today at 2:23 AM	544 bytes	JSON File
Cookies	Today at 2:24 AM	29 KB	Document
Cookies-journal	Today at 2:24 AM	Zero bytes	Document
cowork-enabled-cli-ops.json	Today at 2:23 AM	61 bytes	JSON File
> Crashpad	Apr 13, 2026 at 11:08 AM	--	Folder
> DawnGraphiteCache	Apr 13, 2026 at 11:08 AM	--	Folder
> DawnWebGPUCache	Apr 13, 2026 at 11:08 AM	--	Folder
DIPS	Today at 2:22 AM	37 KB	Document
DIPS-wal	Today at 2:24 AM	8 KB	Document
extensions-blocklist.json	Today at 2:23 AM	177 bytes	JSON File
fcache	Today at 2:23 AM	23 KB	Document
git-worktrees.json	Apr 29, 2026 at 1:20 PM	41 bytes	JSON File

CREATING THE SERVER

- Creating the server requires utilizing the asyncio library, which actually creates the asynchronous servers that provide the actions and lets the Client use them
- When creating an action, the MCP Tool Class is used to define an action
 - All Tools must be created within the same server address, “list_tools()”
 - To use a Tool, all logic of how to use the Tool is in the “call_tool()” address

```
@app.list_tools()
async def tools() -> list[Tool]:
    return [
        Tool(
            name = "diagnosis", # change/add tool to send medicine info, not diagnosis
            description = "Predicts possible illness from symptoms",
            inputSchema = {
                "type": "object",
                "properties": {
                    "symptoms": {
                        "type": "string",
                        "description": "symptoms' description"
                    }
                },
                "required": ["symptoms"]
            }
        ),
    ]
```



```
@app.call_tool()
async def use_tool(name: str, args: dict) -> list[TextContent]:

    if name == "diagnosis":

        symptoms = args["symptoms"]
        prompt = f"<|user|>\nI have {symptoms}. What disease might I have?\n<|assistant|>\n"
        inputs = tokenizer(prompt, return_tensors="pt").to("cpu")

        with torch.no_grad():
            output = model.generate(*inputs, max_new_tokens = 100)

        response = tokenizer.decode(outputs[0], skip_special_tokens=True).split("<|assistant|>")[-1].strip()
        # response = "You might have Influenza."

        return [TextContent(type="text", text=response)]
```

CREATING THE SERVER (THE HARD PART OF IT)

- While creating Tools, defining functions, and programming how to use those Tools are the defining aspects of the server, this part was not the core component
- In order to receive, read, or send information through an MCP Tool, it is extremely difficult
 - Handling information, especially sensitive information, requires gaining access to keys
 - Each time a user uses the Agent with the server, a session is created to remember information (like emails)
 - Parsing websites, extracting information, and making it legible requires navigating through the raw HTML code of a website and making requests with valid identification
 - Encoding/decoding sensitive information
 - Used BeautifulSoup and Regular Expressions

RESULTS

- Fine tuning GPT-2 produced results, with loss ~ 0.2
 - However, overfitting occurred during all 3 fine tunes due to GPT-2's small size and limited training on English
 - Error behind first fine tune's training data
- Llama3 OpenBioLLM also produced promising results, with training & validation loss ~ 0.021

1ST GPT-2 FINE TUNING RESULTS

User: I have a fever and cough. What am I experiencing?

Model: I have a fever and cough. What am I experiencing? (swelled)

Symptoms: vomiting, fatigue, high_fever, headache, swelled_lymph_nodes, malaise, phlegm, throat_irritation, redness_of_eyes, sinus_pressure, runny_nose, congestion, chest_pain, loss_of_smell, muscle_pain

Symptoms: burning_micturition, muscle

2ND GPT-2 FINE TUNING RESULTS

User: I have a cough and fever. What am I experiencing?

Model: I have a cough and fever. What am I experiencing? (vertigo) Paroxysmal Positional Vertigo are vomiting, head ache, nausea, spinning movements, loss of balance, unsteadiness, loss of balance, unsteadiness, unsteadiness, loss of balance, unsteadiness, unsteadiness, loss of balance, unsteadiness, painful walking, loss of balance, unsteadiness, painful loss of appetite, muscle pain, red spots over body, coma

3RD GPT-2 FINE TUNING RESULTS

User: I have a cough and fever. What am I experiencing?

Model: I have a cough and fever. What am I experiencing? I might be experiencing Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Chronic cholestasis, muscle pain. I might be experiencing Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Bronchial Asthma, Bronchial Asthma

User: Why do I have yellow eyes?

Model: Why do I have yellow eyes? I might be experiencing Chronic cholestasis, vomiting, fatigue, weight loss, high fever, yellowish skin, dark urine, abdominal pain. I might be experiencing Jaundice, yellowing of eyes, sinus pressure, stomach bleeding. I might be experiencing Tuberculosis, muscle pain. I might be experiencing Tuberculosis, toxic look (typhos), belly pain. I might be experiencing Tuberculosis, stomach bleeding. I might be

LLAMA3 OPENBIOLLM FINE TUNING RESULTS

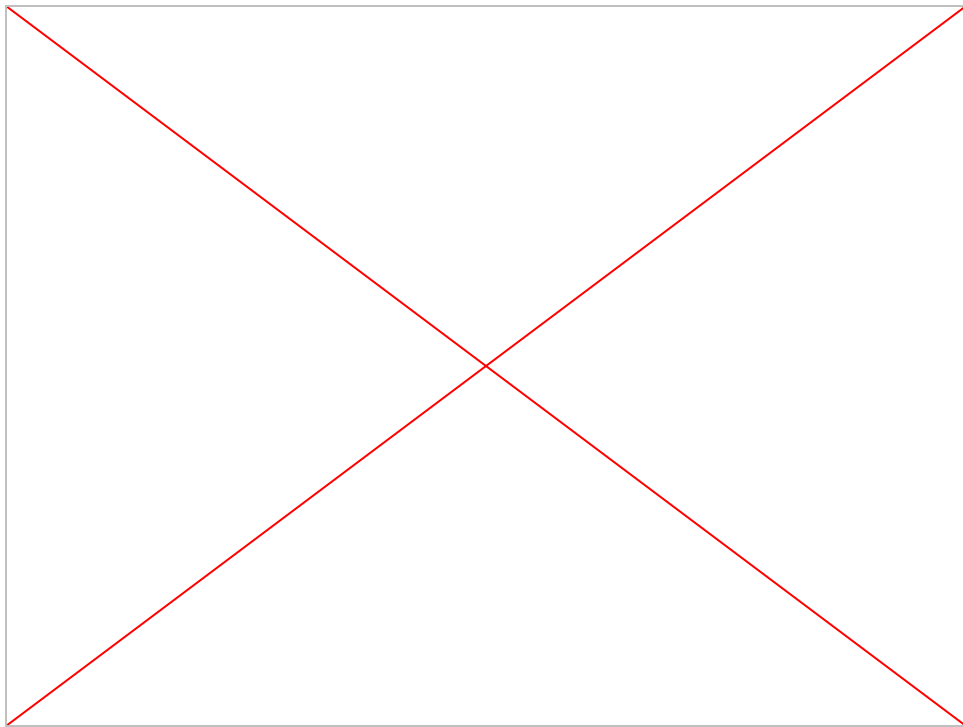
User: I have a fever and cough. What am I experiencing?

Model: I have a fever and cough. What am I experiencing? A) Influenza B) Pneumonia C) Bronchitis D) Asthma

CONCLUSION AND FUTURE WORK

- While the creation/utilization of AI in healthcare is difficult, impacts are highly beneficial and expand possibilities
- MCP could be used to improve the intended tasks/output of the LLM
- Accounting for injuries could prove beneficial by creating accessible medical attention for physical complications
- Future research on preventing overextension
- Potentially utilizing larger biomedical specific LLM
 - Llama3-OpenBioLLM, 70 billion parameters
 - Outperforms GPT-4 and Gemini in this domain!
 - Utilizing Cloud Services (Google Cloud)
- Creating an appropriate UI that would allow for users to input symptoms/feelings, which would be analyzed by the LLM to execute its purpose
- Creating a novel MCP Client for a completely novel AI Agent

DEMO



REFERENCES (LINKS)

- [1] <https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp>
- [2] <https://medium.com/sciforce/step-by-step-guide-to-your-own-large-language-model-2b3fed6422d0>
- [3] <https://huggingface.co/datasets/HuggingFaceFW/fineweb>
- [4] <https://medium.com/@suvarnaptaware/deepseek-step-by-step-guide-48c8fe3be7db>
- [5] <https://www.kaggle.com/datasets/choongqianzheng/disease-and-symptoms-dataset/data>
- [6] <https://blogs.nvidia.com/blog/what-is-a-transformer-model/>
- [7] <https://modelcontextprotocol.io/docs/getting-started/intro>
- [8] <https://www.cms.gov/data-research/statistics-trends-and-reports/national-health-expenditure-data/historical>
- [9] <https://www.medrxiv.org/content/10.1101/2024.07.23.24310847v1>
- [10] <https://pmc.ncbi.nlm.nih.gov/articles/PMC11483254/>
- [11] <https://github.com/modelcontextprotocol/servers>
- [12] <https://www.geeksforgeeks.org/nlp/nlp-how-tokenizing-text-sentence-words-works/>
- [13] <https://www.delveinsight.com/blog/top-artificial-intelligence-based-healthcare-mobile-apps>
- [14] <https://www.nature.com/articles/s41598-024-58760-x>
- [15] <https://pmc.ncbi.nlm.nih.gov/articles/PMC10804900/>
- [16] <https://www.buoyhealth.com/>
- [17] <https://github.com/nateraw/huggingface-datasets-converter>

THANKS!

QUESTIONS?